

```

"""
streamlit_app.py
=====
iPad-first web front end for the AFH acquisition pipeline.

Open a link, see your properties, tap to get the LOI.

Run locally:    streamlit run streamlit_app.py
Hosted:         see DEPLOY_IPAD.md

Import order matters: bootstrap() pushes Streamlit secrets into the
environment, and config.py reads the environment at import time, so bootstrap
has to run before config is imported anywhere.
"""

from __future__ import annotations

import streamlit as st

st.set_page_config(
    page_title="AFH Property Screening",
    page_icon="🏠",
    layout="centered",
    initial_sidebar_state="collapsed",
)

import app_support # noqa: E402
app_support.bootstrap() # noqa: E402 -- must precede the config import

import json # noqa: E402
from datetime import datetime # noqa: E402
from pathlib import Path # noqa: E402
from typing import Any, Optional # noqa: E402

import config # noqa: E402
import deal_calculator # noqa: E402
import gmail_ingestion # noqa: E402
import loi_generator # noqa: E402
import screening # noqa: E402
from models import Listing, ListingAgent, ListingSource # noqa: E402

# -----
# Styling

```

```

# -----

CSS = """
<style>
  .block-container { padding-top: 1.6rem; padding-bottom: 4rem; max-width: 46rem;
/* Touch targets sized for a finger, not a cursor. */
  .stButton > button, .stDownloadButton > button {
    min-height: 3rem; font-size: 1rem; font-weight: 600; border-radius: 10px;
  }
  div[data-testid="stTextInput"] input,
  div[data-testid="stNumberInput"] input { min-height: 2.7rem; font-size: 1rem; }
  .verdict {
    border-radius: 12px; padding: 0.85rem 1.1rem; color: #fff;
    font-weight: 700; font-size: 1.05rem; letter-spacing: 0.01em;
    display: flex; justify-content: space-between; align-items: center;
  }
  .verdict small { font-weight: 500; opacity: 0.9; font-size: 0.8rem; }
  .card {
    border: 1px solid #D9E0E8; border-radius: 12px;
    padding: 0.9rem 1.1rem; margin-bottom: 0.4rem; background: #fff;
  }
  .card h4 { margin: 0 0 0.2rem 0; font-size: 1.02rem; color: #1B2430; }
  .facts { color: #55606E; font-size: 0.86rem; margin: 0; }
  .flag { color: #A32C2C; }
  .mailbtn {
    display: block; text-align: center; background: #1F3A5F; color: #fff !import
    padding: 0.8rem; border-radius: 10px; text-decoration: none; font-weight: 6
  }
</style>
"""

st.markdown(CSS, unsafe_allow_html=True)

def verdict_band(score: Optional[int], threshold: int, sub: str = "") -> None:
    band, label, hexcolor = app_support.verdict(score, threshold)
    st.markdown(
        f'<div class="verdict" style="background:{hexcolor}">'
        f'<span>{band} · {label}</span><small>{sub}</small></div>',
        unsafe_allow_html=True,
    )

# -----
# Access gate
# -----

if not app_support.require_access():

```

```

st.stop()

WS = app_support.workspace()
SETTINGS: dict[str, Any] = WS.setdefault("settings", {})
PROPS: list[dict[str, Any]] = WS.setdefault("properties", [])

COMPANY = app_support.company_from_settings(SETTINGS)
ASSUMPTIONS = app_support.assumptions_from_settings(SETTINGS)
THRESHOLDS = app_support.thresholds_from_settings(SETTINGS)

# -----
# Actions
# -----

def rescreen(record: dict[str, Any]) -> None:
    listing = app_support.listing_from_dict(record["listing"])
    result = screening.screen(listing, THRESHOLDS)
    record["listing"] = listing.to_dict() # screen() sets county
    record["screen"] = {
        "outcome": result.outcome.value,
        "passed": result.passed,
        "reasons": result.reasons,
        "warnings": result.warnings,
    }

def score_with_claude(record: dict[str, Any]) -> Optional[str]:
    """Returns an error message, or None on success."""
    if not config.ANTHROPIC_API_KEY:
        return ("No Anthropic API key. Add ANTHROPIC_API_KEY to your app "
                "secrets, then reload.")
    try:
        import afh_evaluator
        listing = app_support.listing_from_dict(record["listing"])
        assessment = afh_evaluator.evaluate(listing)
        record["assessment"] = assessment.to_dict()
        record.setdefault("rents", {})
        record["rents"].setdefault("phase1", assessment.suggested_phase1_holding_
        record["rents"].setdefault("phase2", assessment.suggested_phase2_full_ren
        return None
    except Exception as exc: # noqa: BLE001
        return f"Scoring failed: {exc}"

def build_loi(record: dict[str, Any]) -> tuple[Optional[Path], Optional[str]]:
    listing = app_support.listing_from_dict(record["listing"])

```

```

assessment = app_support.assessment_from_dict(record.get("assessment"))
rents = record.get("rents", {})

try:
    economics = deal_calculator.build_economics(
        listing, assessment,
        assumptions=ASSUMPTIONS,
        phase1_rent=rents.get("phase1"),
        phase2_rent=rents.get("phase2"),
        strict=False,
    )
except Exception as exc: # noqa: BLE001
    return None, f"Could not build the operating model: {exc}"

try:
    path = loi_generator.generate_loi(
        listing, economics, assessment,
        company=COMPANY,
        purchase_strike_price=record.get("strike_price") or listing.list_pric
    )
    record["loi_path"] = str(path)
    return path, None
except Exception as exc: # noqa: BLE001
    return None, f"Could not generate the PDF: {exc}"

def add_listing(listing: Listing) -> None:
    record = {
        "id": f"{listing.slug}-{int(datetime.now().timestamp())}",
        "listing": listing.to_dict(),
        "added_at": datetime.now().isoformat(timespec="seconds"),
        "rents": {},
    }
    rescreen(record)
    PROPS.insert(0, record)
    app_support.persist()

# -----
# Header
# -----

st.markdown("### AFH property screening")
st.caption("Clackamas County · corporate master lease")

missing_company = not (COMPANY.legal_name and COMPANY.signer_name and COMPANY.ema
if missing_company:

```

```

        st.info("Add your company details on the **Settings** tab before you "
                "generate a letter of intent. They appear on the letterhead.")

tab_props, tab_add, tab_settings, tab_help = st.tabs(
    ["Properties", "Add", "Settings", "Help"]
)

# -----
# Tab: Properties
# -----

with tab_props:
    open_id = st.session_state.get("open_id")

    if open_id and not any(p["id"] == open_id for p in PROPS):
        st.session_state["open_id"] = open_id = None

    # ----- Detail view -----
    if open_id:
        record = next(p for p in PROPS if p["id"] == open_id)
        listing = app_support.listing_from_dict(record["listing"])
        assessment = app_support.assessment_from_dict(record.get("assessment"))
        screen_data = record.get("screen", {})
        score = assessment.afh_feasibility_score if assessment else None

        if st.button("← All properties", use_container_width=True):
            st.session_state["open_id"] = None
            st.rerun()

        verdict_band(score, THRESHOLDS.go_no_go_score,
                     sub=f"threshold {THRESHOLDS.go_no_go_score}")
        st.markdown(f"#### {listing.address}")
        st.caption(
            f"{listing.city}, OR {listing.zip_code or ''} · "
            f"{app_support.money(listing.list_price)} · "
            f"{listing.bedrooms or '?'} bd / {listing.bathrooms or '?'} ba · "
            f"{listing.square_feet or '?'} sq ft · {listing.days_on_market or '?'} "
        )
        if listing.listing_url:
            st.link_button("Open the listing", listing.listing_url,
                          use_container_width=True)

        for warning in screen_data.get("warnings", []):
            st.warning(warning)

        if not screen_data.get("passed", True):

```

```

        st.error(f"Did not pass screening: {screen_data.get('outcome')}")
    for reason in screen_data.get("reasons", []):
        st.write(f"· {reason}")

# ---- score ----
if assessment is None:
    st.markdown("***Not scored yet**")
    if st.button("Score this property", type="primary",
                  use_container_width=True):
        with st.spinner("Reading the listing..."):
            error = score_with_claude(record)
        if error:
            st.error(error)
        else:
            app_support.persist()
            st.rerun()
else:
    with st.expander("What the review found", expanded=True):
        if assessment.pros:
            st.markdown("***In its favour**")
            for item in assessment.pros:
                st.write(f"· {item}")
        if assessment.red_flags:
            st.markdown("***Red flags**")
            for item in assessment.red_flags:
                st.markdown(f'<span class="flag">· {item}</span>',
                             unsafe_allow_html=True)
        st.markdown("***Primary bedroom, shared use**")
        st.write(
            ("Plausible. " if assessment.master_bedroom_shared_potential
             else "Doubtful. ") + assessment.master_bedroom_reasoning
        )
        if assessment.zoning_hoa_risk_notes:
            st.markdown("***Verify with the county and the CC&Rs**")
            for item in assessment.zoning_hoa_risk_notes:
                st.write(f"· {item}")
        if assessment.unknowns:
            st.markdown("***Not stated in the listing**")
            for item in assessment.unknowns:
                st.write(f"· {item}")
        st.caption(f"Rent basis: {assessment.rent_basis}")

    if st.button("Score again", use_container_width=True):
        with st.spinner("Reading the listing..."):
            error = score_with_claude(record)
        if error:
            st.error(error)

```

```

        else:
            app_support.persist()
            st.rerun()

st.divider()

# ---- rent ----
st.markdown("***Rent you are proposing**")
st.caption("Estimates only until you pull real rental comps. Edit freely.")
rents = record.setdefault("rents", {})
col_a, col_b = st.columns(2)
with col_a:
    phase1 = st.number_input(
        f"Phase 1 · months 1-{config.LEASE.phase1_months}",
        min_value=0, step=50,
        value=int(rents.get("phase1") or 0), key=f"p1_{record['id']}"
    )
with col_b:
    phase2 = st.number_input(
        "Phase 2 · operational", min_value=0, step=50,
        value=int(rents.get("phase2") or 0), key=f"p2_{record['id']}"
    )
strike = st.number_input(
    "Purchase option strike price", min_value=0, step=5000,
    value=int(record.get("strike_price") or listing.list_price or 0),
    key=f"strike_{record['id']}"
)

if (phase1, phase2, strike) != (rents.get("phase1"), rents.get("phase2"),
                                record.get("strike_price")):
    rents["phase1"], rents["phase2"] = phase1, phase2
    record["strike_price"] = strike
    app_support.persist()

# ---- pro-forma ----
with st.expander("Operating model"):
    listing_obj = app_support.listing_from_dict(record["listing"])
    economics = deal_calculator.build_economics(
        listing_obj, assessment, assumptions=ASSUMPTIONS,
        phase1_rent=phase1 or None, phase2_rent=phase2 or None,
        strict=False,
    )
    if economics.unverified_inputs:
        st.error(
            "This model is incomplete. Fill in your own figures on the "
            "Settings tab before you rely on any number here."
        )
    st.code(deal_calculator.format_summary(economics), language=None)

st.divider()

```

```

# ---- LOI ----
st.markdown("***Letter of intent**")
if missing_company:
    st.info("Add your company details on Settings first.")
elif phasel <= 0 or phase2 <= 0:
    st.info("Set both rent figures above, then the PDF will build.")
else:
    if st.button("Build the letter of intent", type="primary",
                  use_container_width=True):
        with st.spinner("Writing the PDF..."):
            path, error = build_loi(record)
            if error:
                st.error(error)
            else:
                app_support.persist()
                st.session_state[f"pdf_{record['id']}"] = path.read_bytes()
                st.session_state[f"pdfname_{record['id']}"] = path.name

pdf_bytes = st.session_state.get(f"pdf_{record['id']}")
if pdf_bytes:
    st.download_button(
        "Download the PDF",
        data=pdf_bytes,
        file_name=st.session_state[f"pdfname_{record['id']}"],
        mime="application/pdf",
        use_container_width=True,
    )
    st.caption("Saves to Files. Read it before you send it.")

# ---- email ----
st.markdown("***Email to the listing agent**")
agent_email = st.text_input(
    "Agent email", value=listing.agent.email or "",
    placeholder="not in the listing alert - look it up",
    key=f"email_{record['id']}")
agent_name = st.text_input(
    "Agent name", value=listing.agent.name or "",
    key=f"aname_{record['id']}")

if (agent_email != (listing.agent.email or "")
    or agent_name != (listing.agent.name or "")):
    record["listing"]["agent"]["email"] = agent_email or None
    record["listing"]["agent"]["name"] = agent_name or None
    app_support.persist()

if agent_email:

```



```

import draft_sender
listing.agent.email = agent_email
listing.agent.name = agent_name or None
subject = draft_sender.SUBJECT_TEMPLATE.format(
    address=f"{listing.address}, {listing.city}")
body = draft_sender.build_email_body(
    listing, assessment=assessment, company=COMPANY)

st.markdown(
    f'<a class="mailbtn" href="{app_support.mailto_link(agent_email,
    "Open in Mail</a>", unsafe_allow_html=True)
st.caption("Attach the PDF from Files once Mail opens.")

if app_support.gmail_available():
    if st.button("Put a draft in Gmail instead",
        use_container_width=True):
        try:
            loi_path = record.get("loi_path")
            draft_id = draft_sender.create_draft(
                app_support.gmail_service(), listing,
                Path(loi_path) if loi_path else None,
                assessment=assessment, company=COMPANY,
                dry_run=False)
            st.success(f"Draft {draft_id} is in Gmail, with the PDF "
                "attached. Nothing was sent.")
        except Exception as exc: # noqa: BLE001
            st.error(f"Could not create the draft: {exc}")

with st.expander("Read the message text"):
    st.text_area("Message", value=body, height=340,
        key=f"body_{record['id']}")

st.divider()
if st.button("Delete this property", use_container_width=True):
    PROPS.remove(record)
    app_support.persist()
    st.session_state["open_id"] = None
    st.rerun()

# ----- List view -----
else:
    if not PROPS:
        st.markdown("#### Nothing here yet")
        st.write(
            "Go to the **Add** tab and paste a listing alert email, or type "
            "in a property by hand. Screening runs the moment you add it."
        )

```

```

else:
    scored = [p for p in PROPS if p.get("assessment")]
    st.caption(f"{len(PROPS)} saved · {len(scored)} scored")

    for record in PROPS:
        listing = app_support.listing_from_dict(record["listing"])
        assessment = app_support.assessment_from_dict(record.get("assessment"))
        score = assessment.afh_feasibility_score if assessment else None
        band, label, hexcolor = app_support.verdict(
            score, THRESHOLDS.go_no_go_score)
        passed = record.get("screen", {}).get("passed", True)

        if not passed:
            band, label, hexcolor = "RED", record["screen"]["outcome"].re

    st.markdown(
        f'<div class="verdict" style="background:{hexcolor};'
        f'border-radius:12px 12px 0 0;margin-bottom:0">'
        f'<span>{band} · {label}</span></div>'
        f'<div class="card" style="border-radius:0 0 12px 12px;border'
        f'<h4>{listing.address}</h4>'
        f'<p class="facts">{listing.city}, OR {listing.zip_code or ""'
        f'{app_support.money(listing.list_price)} · '
        f'{listing.bedrooms or "?"} bd · {listing.square_feet or "?"}'
        f'{listing.days_on_market or "?"} DOM</p></div>',
        unsafe_allow_html=True,
    )
    if st.button("Open", key=f"open_{record['id']}",
                use_container_width=True):
        st.session_state["open_id"] = record["id"]
        st.rerun()

# -----
# Tab: Add
# -----

with tab_add:
    st.markdown("#### Add a property")

    mode = st.radio(
        "How?",
        ["Paste a listing alert", "Type it in", "Load a JSON file"],
        label_visibility="collapsed",
    )

    if mode == "Paste a listing alert":

```

```

st.caption(
    "In Mail, select the alert text, copy, and paste below. Works with "
    "Zillow, Redfin and Realtor.com alerts."
)
pasted = st.text_area("Alert text", height=220,
                      placeholder="Paste here...")
use_ai = st.checkbox(
    "Use Claude if the plain reader misses fields",
    value=bool(config.ANTHROPIC_API_KEY),
    disabled=not config.ANTHROPIC_API_KEY,
)
if st.button("Read it", type="primary", use_container_width=True,
            disabled=not pasted.strip()):
    extractor = None
    if use_ai and config.ANTHROPIC_API_KEY:
        try:
            import afh_evaluator
            extractor = afh_evaluator.make_llm_extractor()
        except Exception: # noqa: BLE001
            extractor = None
    listing = gmail_ingestion.parse_listing_from_text(
        pasted, source=ListingSource.MANUAL, llm_extractor=extractor)
    if listing is None:
        st.error(
            "No street address found in that text. Paste more of the "
            "email, or use **Type it in**."
        )
    else:
        add_listing(listing)
        st.success(f"Added {listing.address}. It is on the Properties tab

elif mode == "Type it in":
    with st.form("manual_add"):
        address = st.text_input("Street address *")
        col_a, col_b = st.columns(2)
        city = col_a.text_input("City *")
        zip_code = col_b.text_input("ZIP")
        col_c, col_d, col_e = st.columns(3)
        price = col_c.number_input("Price", min_value=0, step=5000, value=0)
        beds = col_d.number_input("Bedrooms", min_value=0.0, step=1.0, value=
        baths = col_e.number_input("Bathrooms", min_value=0.0, step=0.5, valu
        col_f, col_g, col_h = st.columns(3)
        sqft = col_f.number_input("Sq ft", min_value=0, step=50, value=0)
        dom = col_g.number_input("Days on market", min_value=0, step=1, value
        stories = col_h.number_input("Stories", min_value=0, step=1, value=0)
        url = st.text_input("Listing link")
        description = st.text_area(

```

```

        "Listing description",
        help="Paste the full description. Motivation keywords and the "
            "den/bonus-room check both read from this.",
        height=140)
col_i, col_j = st.columns(2)
agent_name = col_i.text_input("Agent name")
agent_email = col_j.text_input("Agent email")

submitted = st.form_submit_button("Add property", type="primary",
                                   use_container_width=True)

if submitted:
    if not address or not city:
        st.error("Street address and city are both required.")
    else:
        listing = Listing(
            listing_id=f"manual-{int(datetime.now().timestamp())}",
            address=address.strip(), city=city.strip(),
            zip_code=zip_code.strip() or None,
            list_price=int(price) or None,
            bedrooms=beds or None, bathrooms=baths or None,
            square_feet=int(sqft) or None,
            days_on_market=int(dom) or None,
            stories=int(stories) or None,
            description=description, raw_text=description,
            listing_url=url.strip() or None,
            agent=ListingAgent(name=agent_name.strip() or None,
                               email=agent_email.strip() or None),
            source=ListingSource.MANUAL,
        )
        low = description.lower()
        listing.motivation_keywords_found = [
            k for k in config.MOTIVATION_KEYWORDS if k in low]
        add_listing(listing)
        st.success(f"Added {listing.address}.")

else:
    st.caption("Same format as tests/sample_listings.json.")
    uploaded = st.file_uploader("JSON file", type=["json"])
    if uploaded and st.button("Load them", type="primary",
                              use_container_width=True):
        try:
            data = json.loads(uploaded.read().decode("utf-8"))
            records = data["listings"] if isinstance(data, dict) else data
            count = 0
            for rec in records:
                agent = rec.get("agent", {}) or {}

```

```

        listing = Listing(
            listing_id=rec.get("listing_id", rec.get("address", "")),
            address=rec["address"], city=rec.get("city", ""),
            zip_code=rec.get("zip_code"),
            list_price=rec.get("list_price"),
            bedrooms=rec.get("bedrooms"), bathrooms=rec.get("bathroom
            square_feet=rec.get("square_feet"),
            days_on_market=rec.get("days_on_market"),
            stories=rec.get("stories"),
            price_reduction_count=rec.get("price_reduction_count", 0)
            original_price=rec.get("original_price"),
            description=rec.get("description", ""),
            raw_text=rec.get("description", ""),
            listing_url=rec.get("listing_url"),
            agent=ListingAgent(name=agent.get("name"),
                                email=agent.get("email"),
                                phone=agent.get("phone"),
                                brokerage=agent.get("brokerage")),
            source=ListingSource.MANUAL,
        )
        low = listing.description.lower()
        listing.motivation_keywords_found = [
            k for k in config.MOTIVATION_KEYWORDS if k in low]
        add_listing(listing)
        count += 1

        st.success(f"Loaded {count} propert{'y' if count == 1 else 'ies'}")
    except Exception as exc: # noqa: BLE001
        st.error(f"Could not read that file: {exc}")

# ---- Gmail ----
st.divider()
st.markdown("***Pull from Gmail**")
if app_support.gmail_available():
    if st.button("Check for new listing alerts", use_container_width=True):
        with st.spinner("Reading your inbox..."):
            try:
                found = gmail_ingestion.ingest(
                    app_support.gmail_service(), mark_read=False)
                known = {p["listing"]["address"] for p in PROPS}
                added = 0
                for listing in found:
                    if listing.address not in known:
                        add_listing(listing)
                        added += 1

                st.success(f"{len(found)} alert(s) read, {added} new.")
            except Exception as exc: # noqa: BLE001
                st.error(f"Gmail read failed: {exc}")

```

```

else:
    st.caption(
        "Gmail is not connected. The sign-in flow needs a desktop browser, "
        "so you run it once on a computer and paste the token into this "
        "app's secrets. Steps are in DEPLOY_IPAD.md. Pasting alert text "
        "above works without any of that."
    )

# -----
# Tab: Settings
# -----

with tab_settings:
    st.markdown("#### Your company")
    st.caption("This goes on the letterhead and in the signature block.")
    SETTINGS["legal_name"] = st.text_input(
        "Legal name", value=SETTINGS.get("legal_name", ""),
        help="Exactly as it appears on your filed formation documents.")
    SETTINGS["signer_name"] = st.text_input(
        "Who signs", value=SETTINGS.get("signer_name", ""))
    SETTINGS["signer_title"] = st.text_input(
        "Title", value=SETTINGS.get("signer_title", "Owner"))
    col_a, col_b = st.columns(2)
    SETTINGS["phone"] = col_a.text_input("Phone", value=SETTINGS.get("phone", ""))
    SETTINGS["email"] = col_b.text_input("Email", value=SETTINGS.get("email", ""))
    SETTINGS["mailing_address"] = st.text_input(
        "Mailing address", value=SETTINGS.get("mailing_address", ""))

    st.divider()
    st.markdown("#### Screening rules")
    col_c, col_d = st.columns(2)
    SETTINGS["min_bedrooms"] = col_c.number_input(
        "Fewest bedrooms", min_value=1.0, step=1.0,
        value=float(SETTINGS.get("min_bedrooms", 5)))
    SETTINGS["min_dom"] = col_d.number_input(
        "Fewest days on market", min_value=0, step=5,
        value=int(SETTINGS.get("min_dom", 30)))
    col_e, col_f = st.columns(2)
    SETTINGS["min_list_price"] = col_e.number_input(
        "Lowest price", min_value=0, step=25000,
        value=int(SETTINGS.get("min_list_price", 200_000)))
    SETTINGS["max_list_price"] = col_f.number_input(
        "Highest price", min_value=0, step=25000,
        value=int(SETTINGS.get("max_list_price", 900_000)))
    SETTINGS["go_no_go_score"] = st.slider(
        "Score needed for a Go", 50, 95,

```

```

int(SETTINGS.get("go_no_go_score", 75)))

st.divider()
st.markdown("#### Your numbers")
st.warning(
    "Nothing is filled in for you here. A reimbursement rate someone "
    "guessed would flow straight into a rent figure you sign for three to "
    "five years, so the model shows an incomplete-inputs error until these "
    "come from your own rate schedule or remittance advices."
)
SETTINGS["resident_capacity"] = st.number_input(
    "Resident capacity", min_value=1, max_value=5,
    value=int(SETTINGS.get("resident_capacity", 5)),
    help="Oregon adult foster homes are licensed for at most five residents."

st.markdown("***Revenue, per month**")
col_g, col_h = st.columns(2)
SETTINGS["medicaid_resident_count"] = col_g.number_input(
    "Medicaid residents", min_value=0, max_value=5,
    value=int(SETTINGS.get("medicaid_resident_count", 0) or 0))
SETTINGS["medicaid_monthly_rate"] = col_h.number_input(
    "APD rate each", min_value=0, step=50,
    value=int(SETTINGS.get("medicaid_monthly_rate", 0) or 0))
SETTINGS["medicaid_rate_source"] = st.text_input(
    "Where that rate came from, and when",
    value=SETTINGS.get("medicaid_rate_source", ""),
    placeholder="e.g. APD rate schedule pulled 2026-08-20")

col_i, col_j = st.columns(2)
SETTINGS["private_pay_resident_count"] = col_i.number_input(
    "Private pay residents", min_value=0, max_value=5,
    value=int(SETTINGS.get("private_pay_resident_count", 0) or 0))
SETTINGS["private_pay_monthly_rate"] = col_j.number_input(
    "Private rate each", min_value=0, step=100,
    value=int(SETTINGS.get("private_pay_monthly_rate", 0) or 0))
SETTINGS["private_pay_rate_source"] = st.text_input(
    "Where that rate came from",
    value=SETTINGS.get("private_pay_rate_source", ""),
    placeholder="e.g. executed residency agreements")
SETTINGS["room_and_board_monthly"] = st.number_input(
    "Room and board, each Medicaid resident", min_value=0, step=25,
    value=int(SETTINGS.get("room_and_board_monthly", 0) or 0))

st.markdown("***Costs, per month**")
col_k, col_l = st.columns(2)
SETTINGS["staffing_cost"] = col_k.number_input(
    "Staffing", min_value=0, step=250,

```

```

        value=int(SETTINGS.get("staffing_cost", 0) or 0),
        help="Fully loaded: wages, employer taxes, workers comp, benefits, "
            "relief and on-call. Base wage alone understates this badly.")
SETTINGS["food_supplies_cost"] = col_l.number_input(
    "Food and supplies", min_value=0, step=50,
    value=int(SETTINGS.get("food_supplies_cost", 0) or 0))
col_m, col_n = st.columns(2)
SETTINGS["insurance_cost"] = col_m.number_input(
    "Insurance", min_value=0, step=50,
    value=int(SETTINGS.get("insurance_cost", 0) or 0))
SETTINGS["utilities_cost"] = col_n.number_input(
    "Utilities", min_value=0, step=25,
    value=int(SETTINGS.get("utilities_cost", 0) or 0))
SETTINGS["assumptions_effective_date"] = st.text_input(
    "Date you pulled these figures",
    value=SETTINGS.get("assumptions_effective_date", ""),
    placeholder="2026-08-20")

if st.button("Save settings", type="primary", use_container_width=True):
    app_support.persist()
    st.success("Saved.")
    st.rerun()

st.divider()
st.markdown("#### Back up")
st.caption(
    "The hosting disk is wiped whenever the app restarts. Download this "
    "after any change you would not want to redo."
)
st.download_button(
    "Download a backup",
    data=json.dumps(WS, indent=2),
    file_name=f"afh-workspace-{datetime.now():%Y%m%d}.json",
    mime="application/json",
    use_container_width=True,
)
restore = st.file_uploader("Restore from a backup", type=["json"],
                           key="restore")
if restore and st.button("Replace everything with this backup",
                        use_container_width=True):
    try:
        st.session_state["workspace"] = json.loads(restore.read()).decode("utf
        app_support.persist()
        st.success("Restored.")
        st.rerun()
    except Exception as exc: # noqa: BLE001
        st.error(f"Could not read that backup: {exc}")

```



```

# -----
# Tab: Help
# -----

with tab_help:
    st.markdown("#### How this works")
    st.markdown(
        """
        **Add a property.** Paste the text of a listing alert, type one in, or load a
        JSON file. Screening runs immediately: county, bedroom count, price band, and
        whether the seller looks motivated.

        **Score it.** Claude reads the listing and rates feasibility out of 100, with
        the reasoning, the red flags, and a list of what the listing never actually
        said.

        **Set the rent.** The suggested figures are estimates from listing text, not a
        comp analysis. Change them.

        **Build the letter of intent.** Two pages, non-binding, ready to download to
        Files.

        **Send it yourself.** Open in Mail attaches nothing, so add the PDF from Files.
        Nothing is ever sent for you.
        """
    )

    st.markdown("#### Four things worth knowing")
    st.markdown(
        """
        **ZIP codes are not county lines.** 97035, 97070, 97140, 97002, 97062 and
        97236 straddle them. Those pass with a warning instead of an assumption. Check
        the parcel with the county assessor before spending money.

        **Agent emails are rarely in the alert.** Those emails link to the listing;
        they do not include the agent's inbox. Look it up and type it in.

        **The operating model is only as good as your numbers.** It shows an error
        until you enter them on Settings.

        **The letter is non-binding, and Section 10 says so.** Leave that in. This is
        not legal advice – have an Oregon real estate attorney read the template once
        before the first one goes out, and read the lease every time. Ask specifically
        about the guaranteed-rent exposure: as written you owe rent whether or not the
        home is ever licensed and whether or not it ever fills.
        """
    )

```

```
        """"
    )

st.divider()
st.caption(
    f"Model: {config.ANTHROPIC_MODEL} · "
    f"API key {'set' if config.ANTHROPIC_API_KEY else 'missing'} · "
    f"Gmail {'connected' if app_support.gmail_available() else 'not connected'} · "
    f"Saved {WS.get('saved_at') or 'never'}"
)
```